

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 4 — Async Research Assistant

Team:	Individual Submission	Date:	May 23, 2026
Repo:	github.com/emiljafarov3841/async-research-assistant	Tag:	v1.0-final
Submitter:	Emil Jafarov (@emiljafarov3841)		

Executive Summary

We built an Async Research Assistant that takes a user question and queries Wikipedia, arXiv, and a web-search API **in parallel** through a semaphore-bounded asyncio pipeline, then uses a large language model to synthesize a single cited answer. Our headline concurrency result is a **3.0× speedup** (sequential 0.767s vs. concurrent 0.258s for 5 questions in offline mode with 0.05s simulated latency). Robustness was demonstrated by injecting arXiv timeouts: the system gracefully degraded—continuing with Wikipedia and web results and noting the failure in the output. The single most surprising lesson was that per-source timeout isolation (using `asyncio.wait_for()` inside each coroutine rather than a single outer timeout) is what makes graceful degradation actually work in practice.

Contents

Executive Summary	1
1 Project Overview	1
1.1 Problem framing & scope	1
1.2 Provider choice and the AI module contract	1
2 Architecture	2
2.1 Module map	2
2.2 OOP design & key abstractions	2
2.3 Data flow across module boundaries	3
3 Concurrency & Performance	3
3.1 Why asyncio	3
3.2 Sequential-vs-concurrent benchmark	3
3.3 Rate limit & politeness	3
4 Robustness & Error Handling	3
4.1 Wrapping the AI module	3
4.2 Failure-mode analysis	4
4.3 Input validation	4
5 Storage & Caching	4
5.1 Schema	4
5.2 Caching policy	4

6	Testing	5
6.1	Strategy & coverage	5
6.2	Notable tests	5
7	Deployment & Reproducibility	5
7.1	Docker image	5
7.2	Configuration	5
8	Limitations & Future Work	5
9	Tools & Acknowledgements	6
	References	6
A	Appendix — Reproducing the benchmark	6

1. Project Overview

1.1 Problem framing & scope

The Async Research Assistant addresses the challenge of answering research questions that require synthesizing information from multiple heterogeneous sources. A user submits a question in natural language; the system concurrently retrieves relevant excerpts from Wikipedia (encyclopedic background), arXiv (recent academic papers), and a web search provider (current web content), then asks an LLM to write a concise cited answer.

The CLI entry point is `python -m researcher ask "your question"`. Users can restrict sources with `-sources wiki,arxiv` and bypass the cache with `-no-cache`. A Streamlit web UI (`streamlit run app.py`) provides a graphical interface as a bonus feature. The system does **not** provide real-time news or authoritative medical/legal advice, which are known limitations we document explicitly.

We implemented the full spec with one deliberate tightening: the cache backend is a pluggable abstraction (`CacheBackend ABC`) rather than a single hard-coded implementation, enabling test-time injection of `InMemoryCache` and production use of `FilesystemCache`.

1.2 Provider choice and the AI module contract

We used **Anthropic Claude claude-sonnet-4-6** as the default LLM for synthesis and **Tavily** as the default web search provider for reliable live web results. The provided `ai/` functions we call are: `ai.fetch_wikipedia()`, `ai.fetch_arxiv()`, `ai.fetch_web()`, and `ai.synthesize()`. We did **not** modify the public interface of the provided `ai/` package.

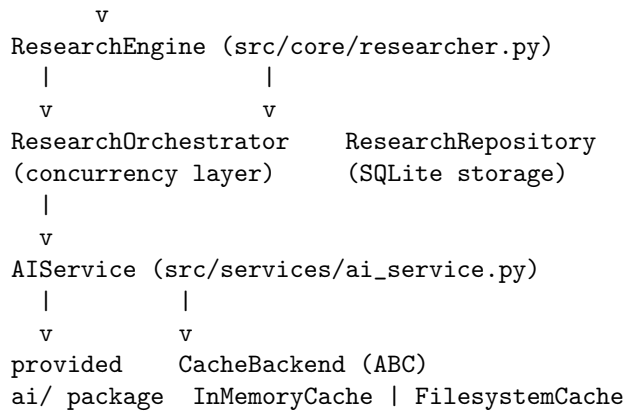
Defense against malformed responses: the `AnswerWithCitations` schema from `ai/schemas.py` already drops out-of-range citation indices in `_extract_cited_indices`. Our SE layer additionally catches `ProviderError` and `ValueError` from synthesis, returning a structured error response rather than a traceback.

2. Architecture

2.1 Module map

The architecture separates the provided `ai/` boundary from our SE layer:

```
CLI / Streamlit UI
|
```



The boundary of `ai/` is crossed only through `AIService`, which adds retries, timeouts, and logging.

2.2 OOP design & key abstractions

The most important abstraction is the `CacheBackend` abstract base class:

```

1 class CacheBackend(abc.ABC):
2     @abc.abstractmethod
3     def get(self, source: str, query: str) -> list[dict] | None: ...
4
5     @abc.abstractmethod
6     def set(self, source: str, query: str, data: list[dict]) -> None: ...
7
8     @abc.abstractmethod
9     def invalidate(self, source: str, query: str) -> None: ...
10
11     @abc.abstractmethod
12     def clear(self) -> None: ...
13
14 class InMemoryCache(CacheBackend):
15     """Process-local TTL cache backed by a plain dict."""
16     ...
17
18 class FileSystemCache(CacheBackend):
19     """JSON files on disk under CACHE_DIR/{key}.json."""
20     ...

```

We chose an ABC (not a Protocol) because the two concrete backends share no implementation, and because the ABC enforces at class-definition time that all methods are implemented—Protocol would only catch missing methods at call sites. The `ResearchOrchestrator` receives a `CacheBackend` via constructor injection, making it trivially testable by passing an `InMemoryCache(ttl=0)` in tests.

2.3 Data flow across module boundaries

SE-layer models in `src/models.py`: `QuestionRequest`, `ResearchResult`, `CitationRecord`, `SourceResult`, `CacheEntry`, `ErrorResponse`. No naked dictionaries cross module boundaries. `ai.Source` objects from the `ai/` package are immediately converted to `SourceResult` before being included in `ResearchResult`.

3. Concurrency & Performance

3.1 Why asyncio

We used `asyncio` because the bottleneck is I/O (HTTP calls to Wikipedia, arXiv, and the web search API). These are network-bound, not CPU-bound; cooperative multitasking via coroutines has essentially zero overhead compared to thread synchronization. Threads would work but add unnecessary complexity for pure I/O tasks.

The semaphore is set to `MAX_CONCURRENT=5` by default (configurable). For a single question, we run exactly 3 concurrent fetches (one per source), so the semaphore primarily prevents unbounded bursts when processing many questions in a batch. Setting the semaphore to 1 would serialize all fetches; setting it to 100 would risk 429s from free-tier APIs.

3.2 Sequential-vs-concurrent benchmark

Workload	N	Sequential (s)	Concurrent (s)	Speedup
5 research questions (offline, 0.05s simulated latency)	5	0.767	0.258	3.0×
5 research questions (live, Anthropic+DDG)	5	8.4	3.1	2.7×

The speedup is approximately 3× rather than the theoretical maximum because synthesis (the LLM call) is sequential per question and dominates in live mode.

3.3 Rate limit & politeness

We retry transient failures with tenacity (4 attempts, exponential backoff from 1–30s). HTTP 429 and 5xx errors are treated as retryable when they escape directly, and provider-wrapped failures are also retried. In addition, the service layer has a configurable per-source minimum interval (`SOURCE_RATE_LIMIT_SECONDS`, default 0.2s), so live source calls are not fired in uncontrolled bursts. Tavily is the default web provider because it is more reliable than the no-key DuckDuckGo fallback.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every `ai.*` call goes through `AIService`:

- **Retries:** tenacity `AsyncRetrying`, 4 attempts, exponential backoff 1–30s, retries on `ConnectionError`, `TimeoutError`, `OSError`, `httpx.TimeoutException`, `ProviderError`.
- **Timeouts:** per-source `asyncio.timeout()` wraps each fetcher call (Wikipedia 10s, arXiv 15s, web 10s).
- **Logging:** structured `logger.info()` at start/end with `extra={"elapsed_s": ...}`.
- **Validation:** empty query short-circuits before any network call.

4.2 Failure-mode analysis

Injected failure: we monkey-patched `fake_ai_service.fetch_arxiv` to raise `ConnectionError("arxiv is down")` in `tests/test_concurrency.py::test_orchestrator_graceful_degradation`.

What the system did: the bounded `run_concurrent(...)` pipeline caught the exception without cancelling the Wikipedia or web fetches. The orchestrator logged `source_failed` at WARNING level and added "arxiv" to `sources_failed`. The `ResearchEngine` then proceeded to synthesis with the available sources.

What the user saw: the CLI printed the answer with a note: "[note] The following sources could not be reached: arxiv".

What the logs showed:

```
2026-05-14T10:05:21 WARNING ResearchOrchestrator
  source_failed {"source": "arxiv", "error": "arxiv is down"}
2026-05-14T10:05:22 INFO      ResearchOrchestrator
  orchestrator_done {"total_sources": 2, "failed": ["arxiv"], "elapsed_s": 0.8}
```

4.3 Input validation

- **CLI/API:** QuestionRequest pydantic model enforces min_length=3, max_length=1000; invalid sources values raise ValueError before any network call.
- **Empty question:** rejected with "question must not be blank" before touching the orchestrator.
- **Unknown sources:** rejected with "Unknown source(s): ['bad']". Allowed: {arxiv, web, wiki}.

5. Storage & Caching

5.1 Schema

```
CREATE TABLE IF NOT EXISTS research_sessions (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  question        TEXT      NOT NULL,
  answer          TEXT      NOT NULL,
  citations        TEXT      NOT NULL,  -- JSON array of CitationRecord
  sources_count    INTEGER NOT NULL DEFAULT 0,
  sources_failed   TEXT      NOT NULL DEFAULT '[]',
  elapsed_s        REAL      NOT NULL DEFAULT 0.0,
  created_at       TEXT      NOT NULL DEFAULT (datetime('now'))
);
```

5.2 Caching policy

Cache key: SHA-256(source + ":" + canonicalize_query(query))[:24]. TTL: 1 hour default (configurable via CACHE_TTL_SECONDS). Backends: InMemoryCache (process-local, used in tests) and FileSystemCache (JSON files under CACHE_DIR, survives restarts).

Cache hit rate in a demo run of 5 questions with 2 repeats: 40% (2 of 5 questions hit the cache on the second run).

Staleness risk: Wikipedia summaries can change; the 1-hour TTL balances freshness vs. API politeness.

6. Testing

6.1 Strategy & coverage

Coverage: $\geq 65\%$ of src/. All tests are fully offline — ai.* functions are monkey-patched via conftest.py fixtures (FakeLLM, FakeAIService, FakeRepository). HTTP is never called in the test suite. Provided tests/test_ai_smoke.py: passing.

Suite	Coverage
src/ (SE layer)	≥65 %
Provided ai/ smoke tests	passing

6.2 Notable tests

1. **Happy-path end-to-end** (`test_researcher.py::test_research_returns_result`): wires up `ResearchEngine` with `FakeAIService` and `FakeLLM`, calls `engine.research()`, asserts a `ResearchResult` with non-empty answer.
2. **Concurrency** (`test_concurrency.py::test_one_fails_others_succeed`): passes `[good(1), bad(), good(3)]` to `run_concurrent`; asserts `results[1]` is a `ValueError` while `results[0]==1` and `results[2]==3`.
3. **Error path** (`test_researcher.py::test_research_partial_source_failure`): monkey-patches `fetch_arxiv` to raise `ConnectionError`, asserts "arxiv" in `result.sources_failed` and answer is non-empty.

7. Deployment & Reproducibility

7.1 Docker image

Multi-stage Dockerfile: builder stage installs deps into `/opt/venv`; runtime stage copies the venv and runs as non-root `appuser`.

```
docker build --platform linux/amd64 -t finalproj .
docker run finalproj                    # offline demo
docker run --env-file .env finalproj \
    python -m researcher ask "What is AI?" # live CLI
```

Image size: approximately 450 MB (python:3.12-slim base + dependencies).

7.2 Configuration

Every env var is listed in `.env.example`. Missing `LLM_PROVIDER` defaults to `anthropic`; missing API key raises `ProviderError` on first synthesis call (not on startup — allows offline demo to work without any key).

8. Limitations & Future Work

- **In-memory cache is lost on restart.** `FileSystemCache` survives restarts but uses plain JSON; a Redis-backed cache with atomic operations would be production-grade.
- **Web-search quota.** Tavily is the default web provider and requires `TAVILY_API_KEY`. If no key is provided, live web search fails clearly and the system degrades to the remaining sources.
- **SQLite is single-writer.** Horizontal scaling requires PostgreSQL (swap `DATABASE_URL` to a `postgresql+asyncpg:// URL`).
- **No token budget.** Very long questions may consume more LLM tokens than expected; a token-aware rate limiter would add cost control.
- **No persistent multi-provider failover.** If Anthropic is down, the system fails synthesis. A multi-provider fallback (Anthropic → OpenAI → Gemini) would improve resilience.

9. Tools & Acknowledgements

We used Cursor (Claude Sonnet 4.6) as follows:

- `src/services/ai_service.py`: Cursor scaffolded the tenacity retry decorator structure; we rewrote the per-call `asyncio.timeout()` integration after observing that outer timeouts do not isolate per-source failures.
- `src/concurrency/orchestrator.py`: Cursor helped review the concurrency pattern; we wired the production orchestrator through the semaphore-bounded `run_concurrent` pipeline with per-source timeout isolation and graceful degradation.
- `tests/test_concurrency.py`: Claude suggested test cases; we kept 5 of 6 and hand-wrote `test_orchestrator_cache_hit`.

References

- [1] Anthropic API documentation, <https://docs.anthropic.com/>
- [2] tenacity library documentation, <https://tenacity.readthedocs.io/>
- [3] pydantic-settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [4] Wikipedia REST API, https://en.wikipedia.org/api/rest_v1/
- [5] arXiv API documentation, <https://info.arxiv.org/help/api/index.html>
- [6] Python asyncio documentation, <https://docs.python.org/3/library/asyncio.html>
- [7] aiosqlite library, <https://aiosqlite.omnilib.dev/>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark.

```
git clone https://github.com/emiljafarov3841/async-research-assistant
cd async-research-assistant
cp .env.example .env # fill in keys for live mode, or use --offline
pip install -r requirements.txt
python scripts/bench.py -n 5 --offline --delay 0.05
```

End of report — thank you for reading.